

Tree

Subordinates

```
#include

using namespace std;
const int maxN = 2e5;

int N, p[maxN+1], sz[maxN+1];
vector G[maxN+1];

void dfs(int u){
    sz[u] = 1;
    for(int v : G[u]){
        dfs(v);
        sz[u] += sz[v];
    }
}

int main(){
    scanf("%d", &N);
    for(int i = 2; i <= N; i++){
        scanf("%d", &p[i]);
        G[p[i]].push_back(i);
    }

    dfs(1);

    for(int i = 1; i <= N; i++)
        printf("%d%c", sz[i]-1, (" \n")[i==N]);
}
```

Tree Matching

```
#include

using namespace std;
const int maxN = 2e5+1;

int N, a, b, cnt;
bool used[maxN];
vector G[maxN];

void dfs(int u = 1, int par = 0){
    for(int v : G[u])
        if(v != par)
            dfs(v, u);

    if(!used[par] && !used[u] && par != 0){
        used[par] = used[u] = true;
        cnt++;
    }
}
```

```

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }
    dfs();
    printf("%d\n", cnt);
}

```

Tree Diameter

```

#include

using namespace std;
const int maxN = 2e5+1;

int N, a, b, last, dist[maxN];
vector G[maxN];

void dfs(int u, int p = 0){
    for(int v : G[u]){
        if(v != p){
            dist[v] = dist[u]+1;
            dfs(v, u);
        }
    }
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs(1);
    for(int i = 1; i <= N; i++)
        if(dist[i] > dist[last])
            last = i;
    dist[last] = 0;
    dfs(last);
    for(int i = 1; i <= N; i++)
        if(dist[i] > dist[last])
            last = i;
    printf("%d\n", dist[last]);
}

```

Tree Distances I

```

#include

using namespace std;
const int maxN = 2e5+1;

int N, a, b, down[maxN], up[maxN], best[maxN];
vector G[maxN];

void dfs1(int u = 1, int p = 0){
    for(int v : G[u]){
        if(v != p){
            dfs1(v, u);
            down[u] = max(down[u], down[v]+1);
        }
    }
}

void dfs2(int u = 1, int p = 0){
    int first = 0, second = 0;
    for(int v : G[u]){
        if(v != p){
            if(down[v] >= down[first]){
                second = first;
                first = v;
            } else if(down[v] > down[second]){
                second = v;
            }
        }
    }

    for(int v : G[u]){
        if(v != p){
            up[v] = max(up[v], up[u]+1);
            if(v == first){
                if(second != 0){
                    up[v] = max(up[v], down[second]+2);
                }
            } else if(first != 0){
                up[v] = max(up[v], down[first]+2);
            }
        }
        dfs2(v, u);
    }

    best[u] = max(up[u], down[u]);
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs1();
    dfs2();

    for(int i = 1; i <= N; i++)
        printf("%d%c", best[i], (" \n")[i==N]);
}

```

Tree Distances II

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, a, b, sz[maxN];
ll down[maxN], up[maxN];
vector G[maxN];

void dfs1(int u = 1, int p = 0){
    sz[u] = 1;
    for(int v : G[u]){
        if(v != p){
            dfs1(v, u);
            sz[u] += sz[v];
            down[u] += down[v] + sz[v];
        }
    }
}

void dfs2(int u = 1, int p = 0){
    if(p != 0)
        up[u] = (up[p]+down[p]) + N - (2*sz[u]+down[u]);
    for(int v : G[u])
        if(v != p)
            dfs2(v, u);
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs1();
    dfs2();

    for(int i = 1; i <= N; i++)
        printf("%lld%c", down[i]+up[i], (" \n")[i==N]);
}
```

Company Queries I

```
#include

using namespace std;
const int maxN = 2e5 + 1;
const int logK = 20;

int N, Q, x, k, p[maxN][logK];

int main() {
    scanf("%d %d", &N, &Q);
    for (int i = 2; i <= N; i++)
```

```

    scanf("%d", &p[i][0]);
    for (int j = 1; j < logK; j++)
        for (int i = 1; i <= N; i++)
            p[i][j] = p[p[i][j - 1]][j - 1];

    for (int q = 0; q < Q; q++) {
        scanf("%d %d", &x, &k);
        for (int i = 0; i < logK; i++)
            if (k & (1 << i))
                x = p[x][i];
        printf("%d\n", x ? x : -1);
    }
}

```

Company Queries II

```

#include

using namespace std;
const int maxN = 2e5+1;
const int logN = 20;

int N, Q, e, a, b, p[maxN][logN];
int timer, in[maxN], out[maxN];
vector G[maxN];

void dfs(int u = 1, int par = 1){
    in[u] = ++timer;
    p[u][0] = par;
    for(int i = 1; i < logN; i++)
        p[u][i] = p[p[u][i-1]][i-1];
    for(int v : G[u])
        if(v != par)
            dfs(v, u);
    out[u] = ++timer;
}

bool ancestor(int u, int v){
    return in[u] <= in[v] && out[u] >= out[v];
}

int lca(int u, int v){
    if(ancestor(u, v)) return u;
    if(ancestor(v, u)) return v;
    for(int i = logN-1; i >= 0; i--)
        if(!ancestor(p[u][i], v))
            u = p[u][i];
    return p[u][0];
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 2; i <= N; i++){
        scanf("%d", &e);
        G[e].push_back(i);
        G[i].push_back(e);
    }

    dfs();

    for(int q = 0; q < Q; q++){

```

```

        scanf("%d %d", &a, &b);
        printf("%d\n", lca(a, b));
    }
}

```

Distance Queries

```

#include

using namespace std;
const int maxN = 2e5+1;
const int logN = 20;

int N, Q, a, b, p[maxN][logN];
int timer, d[maxN], in[maxN], out[maxN];
vector G[maxN];

void dfs(int u = 1, int par = 1){
    in[u] = ++timer;
    d[u] = d[par]+1;
    p[u][0] = par;
    for(int i = 1; i < logN; i++){
        p[u][i] = p[p[u][i-1]][i-1];
    }
    for(int v : G[u])
        if(v != par)
            dfs(v, u);
    out[u] = ++timer;
}

bool ancestor(int u, int v){
    return in[u] <= in[v] && out[u] >= out[v];
}

int lca(int u, int v){
    if(ancestor(u, v)) return u;
    if(ancestor(v, u)) return v;
    for(int i = logN-1; i >= 0; i--){
        if(!ancestor(p[u][i], v))
            u = p[u][i];
    }
    return p[u][0];
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();

    for(int q = 0; q < Q; q++){
        scanf("%d %d", &a, &b);
        printf("%d\n", d[a] + d[b] - 2*d[lca(a, b)]);
    }
}

```

Counting Paths

```
#include

using namespace std;
const int maxN = 2e5+1;
const int logN = 20;

int N, M, a, b, sub[maxN], p[maxN][logN];
int timer, in[maxN], out[maxN];
vector G[maxN];

void dfs1(int u = 1, int par = 1){
    in[u] = ++timer;
    p[u][0] = par;
    for(int i = 1; i < logN; i++){
        p[u][i] = p[p[u][i-1]][i-1];
    }
    for(int v : G[u])
        if(v != par)
            dfs1(v, u);
    out[u] = ++timer;
}

void dfs2(int u = 1){
    for(int v : G[u]){
        if(v != p[u][0]){
            dfs2(v);
            sub[u] += sub[v];
        }
    }
}

bool ancestor(int u, int v){
    return in[u] <= in[v] && out[u] >= out[v];
}

int lca(int u, int v){
    if(ancestor(u, v)) return u;
    if(ancestor(v, u)) return v;
    for(int i = logN-1; i >= 0; i--){
        if(!ancestor(p[u][i], v))
            u = p[u][i];
    }
    return p[u][0];
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs1();

    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        int l = lca(a, b);
        sub[a]++; sub[b]++;
        sub[l]--;
        if(l != 1)
            sub[p[l][0]]--;
    }

    dfs2();

    for(int i = 1; i <= N; i++)
```

```

        printf("%d%c", sub[i], (" \n")[i==N]);
    }

```

Subtree Queries

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, Q, a, b, t, s, timer;
int in[maxN], out[maxN];
ll k[maxN], ds[2*maxN];
vector G[maxN];

void update(int idx, ll val){
    for(int i = idx; i < 2*maxN; i += i&-i)
        ds[i] += val;
}

ll query(int idx){
    ll sum = 0;
    for(int i = idx; i > 0; i -= i&-i)
        sum += ds[i];
    return sum;
}

void dfs(int u, int p = 0){
    in[u] = ++timer;
    for(int v : G[u])
        if(v != p)
            dfs(v, u);
    out[u] = ++timer;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++)
        scanf("%lld", &k[i]);

    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    timer = 0;
    dfs(1);

    for(int i = 1; i <= N; i++)
        update(out[i], k[i]);

    for(int q = 0; q < Q; q++){
        scanf("%d %d", &t, &s);
        if(t == 1){
            update(out[s], -k[s]);
            scanf("%lld", &k[s]);
            update(out[s], k[s]);
        } else if(t == 2){
            printf("%lld\n", query(out[s])-query(in[s]));
        }
    }
}

```

```

    }
}
}

```

Path Queries

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, Q, a, b, t, s;
int timer, in[maxN], out[maxN];
ll x, v[maxN], ds[2*maxN];
vector G[maxN];

void update(int idx, ll val){
    for(int i = idx; i <= timer; i += -i&i)
        ds[i] += val;
}

ll query(int idx){
    ll sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

void dfs(int u = 1, int p = 0){
    in[u] = ++timer;
    for(int c : G[u])
        if(c != p)
            dfs(c, u);
    out[u] = ++timer;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &v[i]);
        for(int i = 0; i < N-1; i++){
            scanf("%d %d", &a, &b);
            G[a].push_back(b);
            G[b].push_back(a);
        }
    }

    dfs();
    for(int i = 1; i <= N; i++){
        update(in[i], v[i]);
        update(out[i], -v[i]);
    }

    for(int q = 0; q < Q; q++){
        scanf("%d %d", &t, &s);
        if(t == 1){
            scanf("%lld", &x);
            update(in[s], -v[s]);
            update(out[s], v[s]);
            update(in[s], x);
            update(out[s], -x);
        }
    }
}

```

```

        v[s] = x;
    } else if(t == 2){
        printf("%lld\n", query(in[s]));
    }
}
}
}

```

Path Queries II

```

#include

/**
 * Link Cut Tree
 */

using namespace std;
const int maxN = 2e5+1;

int N, Q, k, x, y, root, idcounter;

struct Lazy {
    int a = 1, b = 0;
    bool empty(){
        return (a == 1 && b == 0);
    }
};

struct Node {
    Lazy tag;
    bool rev = false;
    Node *c[2] = {nullptr, nullptr}, *p = nullptr;
    int sz, val, sum, mn, mx, id = ++idcounter;

    Node(int v){
        sz = 1;
        val = sum = mn = mx = v;
    }

    void apply(Lazy other){
        mn = mn*other.a + other.b;
        mx = mx*other.a + other.b;
        val = val*other.a + other.b;
        sum = sum*other.a + sz*other.b;
        tag = {tag.a*other.a, tag.b*other.a+other.b};
    }

    void push(){
        if(rev){
            swap(c[0], c[1]);
            if(c[0]) c[0]->rev ^= true;
            if(c[1]) c[1]->rev ^= true;
            rev = false;
        }
        if(!tag.empty()){
            if(c[0]) c[0]->apply(tag);
            if(c[1]) c[1]->apply(tag);
            tag = Lazy();
        }
    }

    void pull(){

```

```

        sum = mn = mx = val;
        sz = 1;
        for(int i = 0; i < 2; i++){
            if(c[i]){
                mn = min(mn, c[i]->mn);
                mx = max(mx, c[i]->mx);
                sum += c[i]->sum;
                sz += c[i]->sz;
            }
        }
    }
} *LCT[maxN];

bool notRoot(Node *t){
    return t->p && (t->p->c[0] == t || t->p->c[1] == t);
}

void rotate(Node *t){
    Node *p = t->p;
    bool b = (p->c[0] == t);
    if((t->p = p->p) && notRoot(p)) t->p->c[(t->p->c[1] == p)] = t;
    if((p->c[!b]=t->c[b])) p->c[!b]->p = p;
    t->c[b] = p;
    p->p = t;
    p->pull();
}

void splay(Node *t){
    while(notRoot(t)){
        Node *p = t->p;
        p->push();
        t->push();
        rotate(t);
    }
    t->push();
    t->pull();
}

Node* access(Node *t){
    Node *last = nullptr;
    for(Node *u = t; u; u = u->p){
        splay(u);
        u->c[1] = last;
        last = u;
    }
    splay(t);
    return last;
}

void evert(Node *t){
    access(t);
    t->rev = true;
}

void link(Node *u, Node *v){
    evert(u);
    u->p = v;
}

void cut(Node *u, Node *v){
    evert(u);
    access(v);
    if(v->c[0] == u) v->c[0]->p = 0;
    v->c[0] = 0;
    v->pull();
}

Node* path(Node *u, Node *v){
    evert(u);

```

```

    access(v);
    return v;
}

Node* LCA(Node *u, Node *v){
    evert(LCT[root]);
    access(u);
    return access(v);
}

bool connected(Node *u, Node *v){
    path(u, v);
    while(v->c[0])
        v = v->c[0];
    return u == v;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x);
        LCT[i] = new Node(x);
    }
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &x, &y);
        link(LCT[x], LCT[y]);
    }

    root = 1;
    for(int i = 0; i < Q; i++){
        scanf("%d", &k);
        if(k == 1){
            scanf("%d %d", &x, &y);
            Node *p = path(LCT[x], LCT[x]);
            p->apply({0, y});
        } else if(k == 2){
            scanf("%d %d", &x, &y);
            Node *p = path(LCT[x], LCT[y]);
            printf("%d ", p->mx);
        }
    }
}

```

Distinct Colors

```

#include

using namespace std;
typedef array triple;
const int maxN = 2e5+1;

int N, a, b, c[maxN], x[2*maxN], ptr[2*maxN], ds[2*maxN], ans[maxN];
int timer, in[maxN], out[maxN];
triple queries[maxN];
vector G[maxN];
map mp;
set S;

void dfs(int u = 1, int p = 0){
    in[u] = ++ timer;
    for(int v : G[u])

```

```

        if(v != p)
            dfs(v, u);
        out[u] = ++timer;
    }

    void update(int idx, int val){
        for(int i = idx; i <= 2*N; i += -i&i)
            ds[i] += val;
    }

    int query(int idx){
        int sum = 0;
        for(int i = idx; i > 0; i -= -i&i)
            sum += ds[i];
        return sum;
    }

    int main(){
        scanf("%d", &N);
        for(int i = 1; i <= N; i++){
            scanf("%d", &c[i]);

            for(int i = 0; i < N-1; i++){
                scanf("%d %d", &a, &b);
                G[a].push_back(b);
                G[b].push_back(a);
            }
            dfs();

            for(int i = 1; i <= N; i++){
                x[in[i]] = x[out[i]] = c[i];

                for(int i = 2*N; i > 0; i--){
                    if(mp[x[i]])
                        ptr[i] = mp[x[i]];
                    mp[x[i]] = i;
                }

                for(int i = 1; i <= 2*N; i++){
                    if(S.count(x[i]) == 0)
                        update(i, 1);
                    S.insert(x[i]);
                }

                for(int i = 1; i <= N; i++){
                    queries[i-1] = {in[i], out[i], i-1};
                }
                sort(queries, queries+N);

                int l = 1;
                for(int i = 0; i < N; i++){
                    while(l < queries[i][0]){
                        if(ptr[l])
                            update(ptr[l], 1);
                        l++;
                    }
                    ans[queries[i][2]] = query(queries[i][1]) - query(queries[i][0]-1);
                }

                for(int i = 0; i < N; i++){
                    printf("%d%c", ans[i], (" \n")[i==N-1]);
                }
            }
        }
    }

```

Finding a Centroid

```
#include

using namespace std;
const int maxN = 2e5+5;

int N, a, b, p[maxN], sz[maxN];
vector G[maxN];

void dfs(int u = 1){
    sz[u] = 1;
    for(int v : G[u]){
        if(v != p[u]){
            p[v] = u;
            dfs(v);
            sz[u] += sz[v];
        }
    }
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();

    for(int i = 1; i <= N; i++){
        bool centroid = true;

        if(p[i] != 0 && N-sz[i] > N/2)
            centroid = false;

        for(int v : G[i])
            if(v != p[i] && sz[v] > N/2)
                centroid = false;

        if(centroid){
            printf("%d\n", i);
            return 0;
        }
    }
}
```